

Design and Analysis of Algorithms

- *Course Code:* BCSE304L
- *Course Type:* Theory (ETH)
- *Slot:* A1+TA1 & & A2+TA2
- *Class ID:* VL2023240500901
VL2023240500902

A1+TA1

Day	Start	End
Monday	08:00	08:50
Wednesday	09:00	09:50
Friday	10:00	10:50

A2+TA2

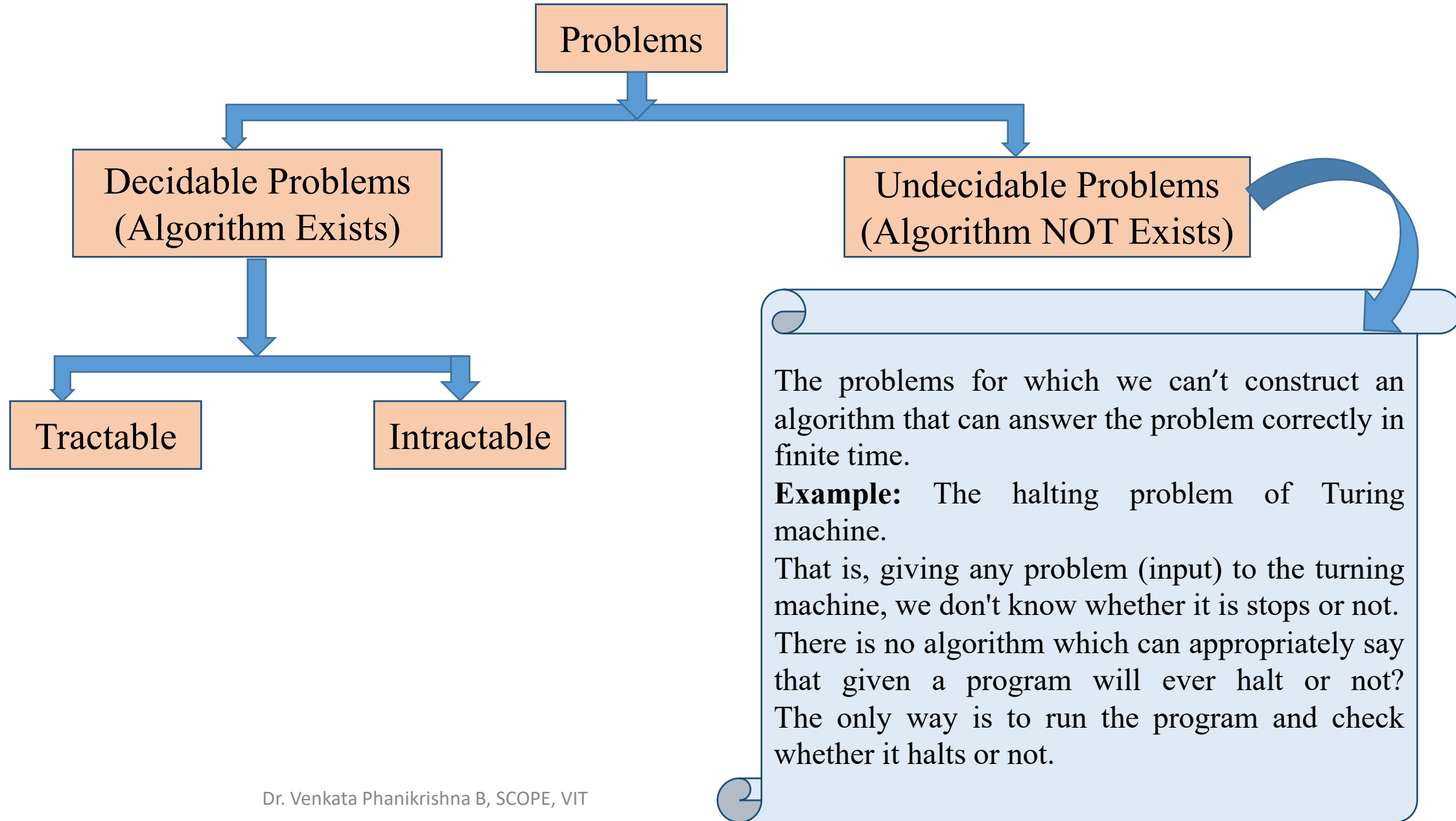
Day	Start	End
Monday	14:00	14:50
Wednesday	15:00	15:50
Friday	16:00	16:50

Syllabus- Module 7

Module:7	Classes of Complexity and Approximation Algorithms	8 hours
-----------------	---	----------------

The Class P - The Class NP - Reducibility and NP-completeness – SAT (Problem Definition and statement), 3SAT, Independent Set, Clique, Approximation Algorithm – Vertex Cover, Set Cover and Travelling salesman

Tractable and Intractable Problems



Tractable

Problems with solution time-bound by a polynomial time. Or, there exists at least one polynomial bound algorithm called tractable problems.

That is, problems could solve in $O(n^k)$

Ex: Ordered Search ($O(\log n)$),
Polynomial evaluation $O(n)$
Sorting $O(n \log n)$

Advantage of Tractable algorithm: It is to write them in a program and then run them. In the software industry or in real-time, whatever programs we see on our computers are tractable.

Note: Even a program which is having $O(n^{1000})$ is a tractable or easy problem or Fast Algorithm. Because, even though any problem shows $O(n^{1000})$ initially, with experience, gradually over a period of time, there have been many solutions proposed that are actually bringing the complexity to a smaller number.

Intractable

Problems with solution times not bound by polynomial are hard or intractable problems

Ex:

Traveling Sales Person $O(n^2 2^n)$
Knapsack $O(2^{n/2})$

Which means, if you are not able find out any algorithm which polynomial time and running $O(C^n)$

Note: Intractable problem programs would run so slowly, and people would never like to run them.

Optimization/Decision Problems

Optimization Problems:

- These problems aim to find the best solution among a set of feasible solutions. The goal is typically to maximize or minimize an objective function.
- **Example:** The Traveling Salesman Problem (TSP) seeks to find the shortest possible route that visits each city exactly once and returns to the original city. TSP problem whose minimum length path is 10 (that is an **optimal problem**)

Decision Problems:

- These problems involve answering a yes/no question about the existence of a solution that satisfies certain criteria.
- Example: In the TSP context, a decision problem could be whether there exists a route with a total length less than or equal to a given threshold. Is there any solution whose weight is at most 15 (that is the **decision problem**)?

Relationship between optimization and decision problems

The relationship between optimization and decision problems is crucial in computational complexity theory, particularly concerning tractability and intractability:

- **Relation between Complexity:**

- If an optimization problem is **easy**, meaning it can be solved in polynomial time, then the associated decision problem (e.g., determining if a solution exists below a certain threshold) is also **easy**.
- Conversely, if the decision problem is intractable (i.e., hard), meaning there's no known polynomial-time algorithm to solve it, then the optimization problem is definitely intractable.

- **Focus on Decision Problems:**

- The theory of NP-Completeness primarily revolves around decision problems. This is because proving a decision problem to be hard (NP-complete) is sufficient to demonstrate that the corresponding optimization problem is also hard.

Verification algorithm:

- **Verification Algorithm:** A verification algorithm is an algorithm that, given the input instance of the problem and the proposed solution, can efficiently (in polynomial time) verify whether the solution is correct. This means that the correctness of the solution can be efficiently checked or verified in polynomial time.
- **Polynomial Verification Algorithm:** If such a verification algorithm exists and it runs in polynomial time, then the decision problem is said to have a polynomial-time verification algorithm.

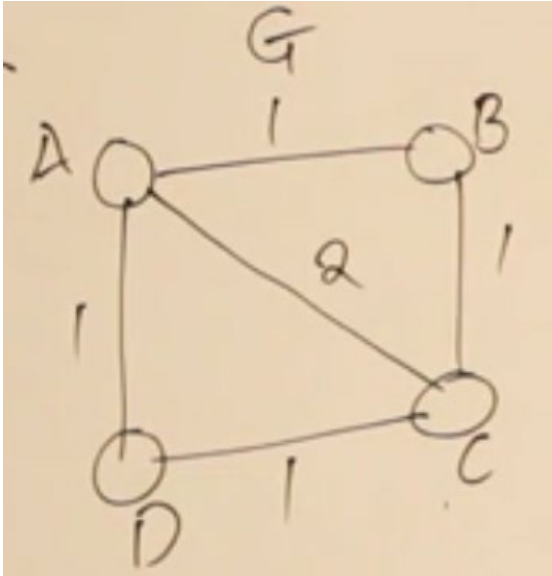
The existence of a polynomial-time verification algorithm implies that the problem is verifiable in a practical sense. We can quickly check if a solution is valid without having to solve the problem from scratch.

Note:

- **Decision Problem:** You have a decision problem where the answer is either YES or NO.
- **Solution:** Someone claims to have a solution to the problem.

Verification algorithm:

Given a decision problem and solution, and verifying the solution in polynomial time then we can say that this particular problem has a polynomial verification algorithm.



TSP — almost 5

A-B-C-D-A

Importance of Verification Algorithms (Optional)

- Verification algorithms play a crucial role in various applications like cryptography, code correctness, and automated theorem proving.
 - In cryptography, verification algorithms are used to confirm the authenticity of digital signatures. The verifier receives a message, a signature, and a public key. The verification algorithm efficiently checks if the signature was indeed generated using the corresponding private key.
 - In code correctness, verification algorithms can be used to check if a program's output matches the expected behavior for specific inputs.
 - Automated theorem proving systems often rely on verification algorithms to confirm the validity of derived theorems within a formal system.
- This concept is particularly relevant in the context of **NP problems**. NP stands for "**nondeterministic polynomial time**."

P-Class

- The class P consists of problems solvable in polynomial time.
 - This means the time to solve the problem grows proportionally to the size of the input, raised to some constant power (e.g., n^2 , n^3). We use Big O notation (**$O(n^k)$**) to represent this, where k is a constant.
- These problems are considered **tractable** because efficient algorithms exist to solve them in polynomial time. This makes them suitable for solving with computers even for large datasets.

Examples:

- Finding the minimum or maximum element in an array.
- Sorting a list of numbers.
- Finding the shortest path between two nodes in a graph (using algorithms like Dijkstra's algorithm).
- Checking if a string is a palindrome.
- Checking if a number is prime, searching a sorted list.

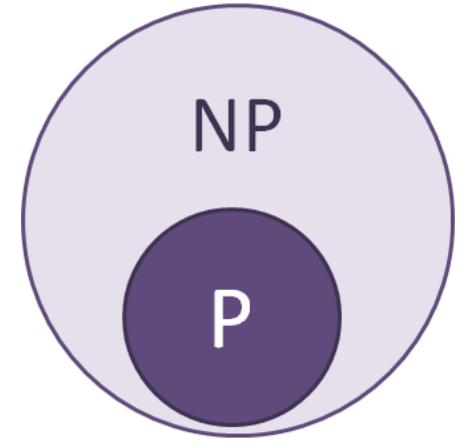
Class NP

- The class NP consists of problems where **solutions** can be **verified in polynomial time**.
 - Even if finding the solution is difficult, checking if a proposed solution is correct is fast.
- **Examples:** Traveling Salesman Problem (TSP) - verifying a proposed route is quick, but finding the shortest route might be difficult.

Every decision problem that is solvable by a deterministic polynomial time algorithm is also solvable by a polynomial time non-deterministic algorithm

Note: Our understanding of NP problems remains uncertain. We cannot definitively determine whether they can be solved in polynomial time or not. Take, for instance, the Traveling Salesman Problem (TSP), which is known to be exponential in nature. Despite extensive efforts, we have yet to discover a polynomial-time solution for TSP. However, it's crucial to acknowledge that we cannot dismiss the possibility that a polynomial-time algorithm may emerge in the future, or conversely, that someone may prove the non-existence of such an algorithm. Therefore, our current knowledge about these problems is limited, and we classify them as NP problems.

Relation between P and NP

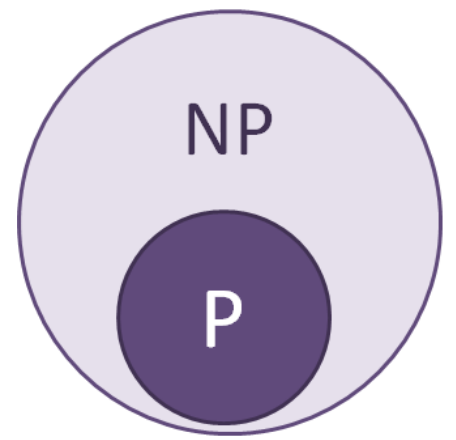


Problems in “P” quickly (or easily) solved
{Because we have polynomial time algorithm to solve them}

Problems in “NP” quickly (or easily) Verified
{But we can really don’t know whether it could be solved easily or not}

NP-P $\rightarrow$ intractable

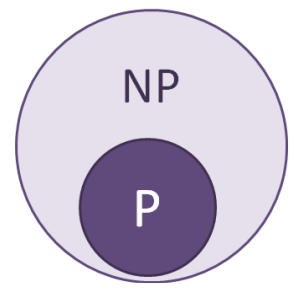
Relation between P and NP



There are three possibilities:

- $P = NP$: All NP problems are solvable efficiently.
- $P \subset NP$: There are some NP problems that cannot be solved efficiently (most believe this is true).
- $P \supset NP$: All NP problems can be solved efficiently, but there are also problems easier than those in NP. (This is considered unlikely)

Relation between P and NP

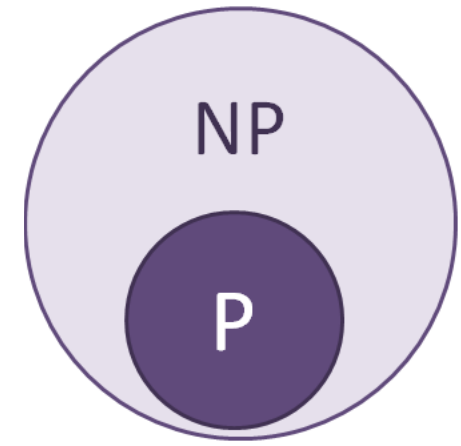


If a problem is in P, we can efficiently find the solution itself in polynomial time. Since verification is a simpler task than finding the solution, we can also verify a proposed solution in polynomial time. This makes all P problems automatically fall under NP as well.

Whether every problem whose solution can be verified quickly can also be solved quickly.

Open Questions:

- Proving $P = NP$: This would require showing efficient solutions for all problems in NP, which is practically impossible due to the vast number of problems.
- Proving $P \subset NP$: This would require finding a single problem in NP that cannot be solved efficiently. This is also a challenging task.



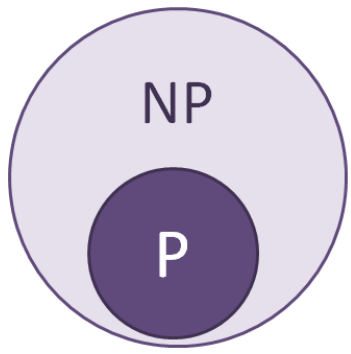
Directly proving $P=NP$ is impossible. **Why?**

Because, in universe there are millions of problems, solving all problems is impossible.

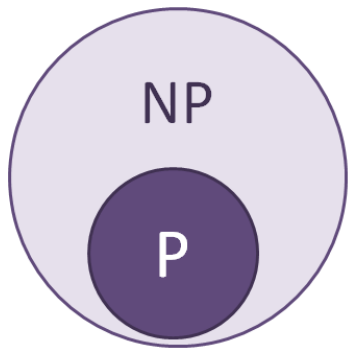
Consider the analogy of determining the world's Grand Master in chess. To earn this title, one would theoretically need to play against every chess player globally, which is practically impossible due to the millions of players worldwide.

What we could do:

- A more feasible approach involves identifying champions from each country who have defeated all opponents within their respective nations.
- These champions then compete in a tournament, with the winner being declared the Grand Master.
- This structured approach reduces the complexity of the task by focusing on representative individuals rather than attempting to engage with every player individually.



- Similarly, attempting to solve every problem to determine their polynomial-time solvability is unfeasible.
- Instead, the concept of **Reducibility** is employed.
 - We will take one problem (Some particular problem), which could be (or which is) as difficult as remaining all problems in NP.
 - And then, If we solving that problem in polynomial time Then we declare that, since this problem (complicated problem) is solvable in polynomial time, So all the problems could solvable in polynomial time.



- Similarly, attempting to solve every problem to determine their polynomial-time solvability is unfeasible.
- Instead, the concept of **Reducibility** is employed.
 - By demonstrating that a single problem, often an NP-complete problem, can be efficiently solved in polynomial time, it implies that all problems in NP could also be solved efficiently.
 - Thus, rather than proving every problem's solvability individually, reducibility allows us to establish the complexity relationship between different classes of problems, offering insight into the broader landscape of computational complexity theory.

Reducibility or Polynomial time Reduction

- A problem “AA” is said to be polynomial time reducible to a problem “BB” if
 1. Every instance ‘ α ’ of “AA” can be transformed to some instance ‘ β ’ in polynomial time.
 2. Answer to ‘ α ’ is **YES** if and only if (iff) answer to ‘ β ’ is **YES**

Significance of this is:

In case, if we have a polynomial time algorithm to solve ‘ β ’, i.e., $O(n^j)$

If we convert the problem AA (i.e., ‘ α ’) to problem BB (i.e., ‘ β ’) in polynomial time, i.e., $O(n^k)$.

Therefore, in order to solve the problem AA what we found out is

1st convert α , which time taken is $O(n^j)$ + and then solve the β which time taken is $O(n^k)$

i.e., $O(n^j) + O(n^k) \rightarrow$ Polynomial time.

Then

‘BB’ is easy then ‘AA’ is also easy. (i.e., solve in polynomial time)

If ‘AA’ is not in P then BB is definitely not in P

Reducibility or Polynomial Time Reduction

A problem "AA" is said to be polynomial time reducible to a problem "BB" if every instance ' α ' of "AA" can be transformed into some instance ' β ' in polynomial time. Furthermore, the answer to ' α ' is "YES" if and only if the answer to ' β ' is also "YES"

The significance of this concept lies in its implications for problem complexity. If there exists a polynomial time algorithm to solve problem "BB" (denoted as $O(n^j)$), and we can convert problem "AA" (instance ' α ') to "BB" (instance ' β ') in polynomial time (denoted as $O(n^k)$), then solving problem "AA" becomes a two-step process:

1. Converting ' α ' to ' β ', which takes $O(n^j)$ time.
2. Solving ' β ', which takes $O(n^k)$ time.

Combining these steps results in a polynomial time complexity: $O(n^j) + O(n^k) = O(n^{\max(j, k)})$.

- ✓ This implies that if problem "BB" is easy to solve (i.e., solvable in polynomial time), then problem "AA" is also easy to solve.
- ✓ Conversely, if "AA" is not solvable in polynomial time (i.e., not in P), then "BB" cannot be in P either.
- ✓ This relationship between problems under polynomial time reduction helps classify problems based on their computational complexity.

NP-Hard

- **NP** → Set of all algorithms which could be verified in polynomial time. (So this set even contains P (i.e., polynomial-time) Algorithms.
- **NP-Hard** → A problem is NP-Hard if every problem in the NP class (problems solvable by a non-deterministic Turing machine in polynomial time) can be reduced to it in polynomial time using reductions.
- In simpler terms If every problem in NP can be polynomial-time reducible to a problem "A," then "A" is called NP-Hard.
 - **Or** A problem is **NP-hard** if all problems in NP are polynomial time reducible to it, even though it **may not be in NP** itself.

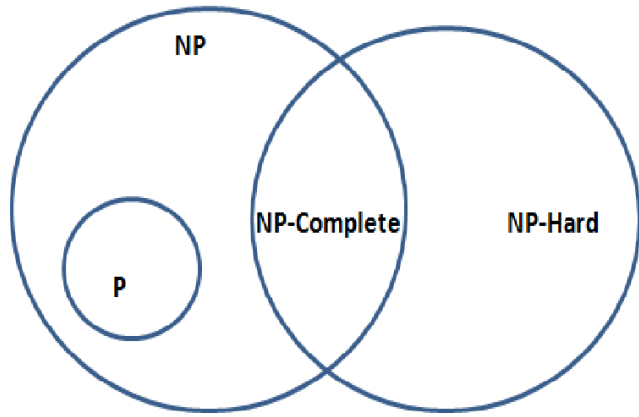
Implication : In order to prove that every problem in NP can be solved in polynomial time, you need to just prove that you could find a polynomial-time algorithm to "A." Therefore, instead of researching all these problems (i.e., NP class problems), you could take only one such problem and work on it.

NP-Complex

- **Text book Definition:** A problem that is NP-Complete can be solved in polynomial time if and only if (iff) all other NP-Complete problems can also be solved in polynomial time.
- If problem "A" is in NP and NP-Hard, then it is called as NP-Complex. (Means that problem "A" happens to be present in NP set itself).

or

- If "A" is NP-Hard and "A" is converted to "B" in polynomial time, then "B" becomes as hard as any problem in NP (i.e., NP-Hard), and If "B" is already known as NP, then B is NP-complete.



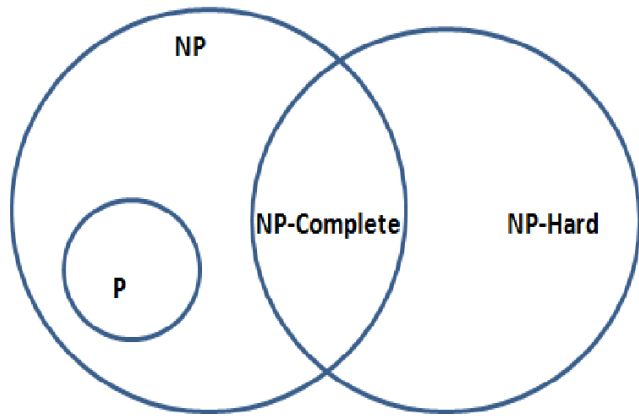
From the 1970s or 71's to this year, scientists/Researchers started to believe it. (Means, we don't have any correct proof.)

What is the importance of having such a problem (NP-Complete) is:

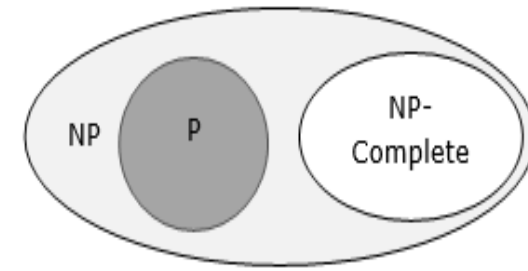
If you want to test or do some research whether $P=NP$, we need not solve every problem. You just take one problem, which is known to be NP-Complete, and from that problem, you prove that you can solve it in polynomial time. Then it is sufficient to prove $P=NP$.

(or)

You take one NP-Complete problem (because NP-Complete problem is already present in NP), if you prove that it can never be solved in polynomial time, then you have to find out at least one problem, which is present in NP and which is not present in P then we completed the $P \neq NP$ prove.



Note:



NP-Hard or **NP-Complete** problem is solved in polynomial time, then $NP=P$
NP problem or **NP-complete** is proven to be not solvable in polynomial time, then $P \neq NP$.

- Easy $\rightarrow \mathcal{P}$
- Medium $\rightarrow \mathcal{NP}$
- Hard $\rightarrow \mathcal{NP-Complete}$
- Hardest $\rightarrow \mathcal{NP-Hard}$

Polynomial Algorithms

- All basic mathematical operations; addition, subtraction, division, multiplication
- Testing for primacy (demonstrating how our decision making process is flawed)
- Hashtable lookup, string operations, sorting problems
- Shortest Path Algorithms; Djikstra, Bellman-Ford, Floyd-Warshall
- Linear and Binary Search Algorithms for a given set of numbers

Examples of NP problems:

- Traveling salesman problem
- satisfiability problems
- graph-covering problems
- etc

Examples NP-Hard Problems

- The circuit-satisfiability problem
- Set Cover
- Vertex Cover
- Travelling Salesman Problem

Examples of NP problems:

- **Traveling salesman problem**
- satisfiability problems
- graph-covering problems
- etc

Examples NP-Hard Problems

- The circuit-satisfiability problem
- Set Cover
- Vertex Cover
- **Travelling Salesman Problem**

Examples NP-Complete Problems

Circuit-satisfiability problem

Travelling Salesman Problem

- Boolean satisfiability problem (SAT)
- Knapsack problem
- Hamiltonian path problem
- Travelling salesman problem (decision version)
- Subgraph isomorphism problem
- Subset sum problem
- Clique problem
- Vertex cover problem
- Independent set problem
- Dominating set problem
- Graph coloring problem

NP-Hard but not NP-Complete

- NP-Complete problems are the subset of NP-Hard for which solutions are verifiable in polynomial time, and the NP-Hard problems are the ones that are at least as hard as any in NP. So, **which solutions cannot be verified** in polynomial time are NP-Hard but not NP-Complete.

For example, in case of Subset Sum problem

Enumeration (mentioning a number of things one by one.) of Subset Sum problem is NP-hard.
How?

- The Subset Sum problem asks: given a set S of numbers, is there a subset that sums to a given value X ? To verify a solution, you give the subset, and then someone can add them up and verify that they sum to the target. So, it's easy to verify a solution. The subset is never bigger than S , so you can verify in no more than $O(|S|)$ time.
- Given a set S of numbers, **how many subsets of S sum to a given value X ?** It's at least as hard as Subset Sum

How can we identify all these set of NP-Complete Problem?

- One simple Proof: Find out one problem which is known to be NP-Complete and from that problem give Transformation to some other problem in NP, then that NP problem become NP-Complete.

Importance of saying a problem is NP-Complete is:

To prove $P=NP$, without solving all problems in NP in polynomial time, if any NP-complete problem can be solved in polynomial time, then every problem in NP has a polynomialtime algorithm.

Any

Question



PresenterMedia



Thank You!

**FOR YOUR
ATTENTION**

